O&M runbooks — steady-state operations & maintenance

Operations and maintenance for a **deployed** Claude apps gateway. `greenfield-deployment.md` covers the initial deploy; this document covers what you do afterwards: rotating the certificate and secrets, refreshing the RDS CA bundle, pushing new Claude Code / image versions, responding to alarms, backup/restore, and teardown. Symptom-first diagnosis lives in `troubleshooting.md`.

Every command uses the repo's own scripts and `deploy.env` variables — never hardcoded org values. Run operator commands from a host that has `deploy.env` filled in and AWS credentials for `us-gov-west-1` (the scripts source `scripts/common.sh`, which loads `deploy.env` and exports `AWS_REGION`). The three stack names come from `deploy.env`:

- `DB_STACK_NAME` — `01-database.yaml` (default `${NAME_PREFIX}-db`)
- `GATEWAY_STACK_NAME` — `02-gateway.yaml` (default `${NAME_PREFIX}`)
- `OBS_STACK_NAME` — `03-observability.yaml` (default `${NAME_PREFIX}-obs`)

Accepted risks and design trades (surfaced up front)

Two deliberate, SSP-scoped decisions affect operations and are stated here so they are not discovered mid-incident:

- **Telemetry collector is a localhost sidecar, not a service.** The gateway rejects a non-HTTPS telemetry forward URL off localhost, so the ADOT collector runs **inside the gateway task**: loopback within one Fargate network namespace, no network telemetry hop to secure, and the collector shares the gateway's lifecycle. Operationally the collector is **not** a separate service to restart or roll on its own — see runbook 6. Two consequences of the default `TELEMETRY_FAIL_CLOSED=true` (AU-5): **a persistently failed or unhealthy collector stops the gateway task** (ECS replaces it; check the `otel/` log streams and the collector health check before suspecting the gateway itself), and every task stop drains gateway-first / collector-last with up to a 120 s flush window, so task stops during rotations and deploys take up to ~2 minutes longer than they otherwise would.
- **S3 Object Lock deferred.** The activity archive and ALB-log buckets rely on `DeletionPolicy: Retain` + bucket lifecycle, not Object Lock. A privileged operator can still delete archived objects; there is no WORM guarantee.
- **Bedrock prompt-logs bucket: delivery grant is bucket-wide.** The prompt-logs bucket policy grants `bedrock.amazonaws.com` `s3:PutObject` on the whole bucket rather than AWS's documented `AWSLogs/<account>/...` example prefix, because the large-body delivery prefix is delivery-managed and undocumented; the `aws:SourceAccount` / `aws:SourceArn` conditions carry the cross-account restriction. Tighten to the observed prefixes once delivery is in use in a deployment — runbook 11.

---

1. ALB TLS certificate rotation

Trigger / Frequency: The `${NAME_PREFIX}-certificate-expiry` CloudWatch alarm fires (`AWS/CertificateManager` `DaysToExpiry` ≤ `CERT_EXPIRY_ALARM_DAYS`, default 30) — or any unplanned re-issue (key compromise, CA change). **Imported ACM certificates do NOT auto-renew**, so this is a scheduled human task, roughly once per certificate lifetime (typically annually).

Preconditions:

- Access to the enterprise CA to sign a new leaf (serverAuth EKU).
- `CERTIFICATE_ARN` and `GATEWAY_FQDN` set in `deploy.env` (the current cert's ARN — rotation replaces the certificate *in place* under the same ARN, so no stack update is needed and the ALB keeps its DNS name).
- Run the key/CSR steps on the PKI workstation; `import-enterprise-cert.sh` sources `common.sh` with `COMMON_SH_OPTIONAL_ENV=1`, so it works without a filled-in `deploy.env` (it only needs `set_env_var`, which no-ops with a warning if `deploy.env` is absent).

Steps (exact commands):

1. Generate a fresh key + CSR. The script does this under `umask 077` and removes any pre-existing key first, so the key is never briefly world-readable:

```
bash ./scripts/import-enterprise-cert.sh csr "$GATEWAY_FQDN" # writes ${GATEWAY_FQDN}.key.pem (0600) + ${GATEWAY_FQDN}.csr # SAN is exactly DNS:${GATEWAY_FQDN} (the corporate CNAME, not the ALB name) # Key type defaults to EC P-256; append rsa2048 (or rsa3072) if the CA # only issues RSA: ...csr "$GATEWAY_FQDN" rsa2048
```

1. Submit `${GATEWAY_FQDN}.csr` to the enterprise CA (serverAuth EKU). Collect the new leaf (`leaf.pem`) and the CA chain (`chain.pem`, intermediates first, root last).

2. **Publish the new SHA-256 fingerprint to developers BEFORE cutting over.** Rotation re-triggers Claude Code's first-connect trust prompt; developers who pinned the old fingerprint must be told the new one. The import step prints it, but you can print it ahead of the cutover from the leaf:

```
bash openssl x509 -in leaf.pem -noout -fingerprint -sha256
```

1. Replace the certificate **in place** under the existing ARN — the ALB listener picks up the new material with **no stack update**:

```
bash ./scripts/import-enterprise-cert.sh import "$GATEWAY_FQDN" \ leaf.pem "${GATEWAY_FQDN}.key.pem" chain.pem \ --certificate-arn "$CERTIFICATE_ARN"
```

The script re-validates SAN, serverAuth EKU, and key↔cert match before importing; on success it re-prints the fingerprint and the new expiry date, and persists `CERTIFICATE_ARN` back into `deploy.env` (unchanged on an in-place replace).

Verification:

- `scripts/verify-gateway.sh` — step 2/3 fetches the served cert, cross-checks its SHA-256 against the ACM-imported cert (when AWS creds are available), and prints the fingerprint to publish. A mismatch flags TLS interception (ZIA inspection), not a rotation success.
- Confirm the alarm clears: after ACM re-computes `DaysToExpiry` (daily, `Period: 86400`), `${NAME_PREFIX}-certificate-expiry` returns to `OK`.

Rollback / recovery: Re-import the previous leaf/key/chain under the same `--certificate-arn` (keep the outgoing material until the new cert is confirmed live). Because the ARN is stable, rollback is another in-place `import`; the ALB never changes DNS name or listener config.

Notes & pitfalls:

- **Do not** delete-and-recreate the ACM certificate or change the listener's `CertificateArn` in the template to rotate — that risks an ALB/listener update. In-place replace under the same ARN is the only sanctioned path.
- The SAN must be `DNS:${GATEWAY_FQDN}` (the corporate CNAME), never the `*.elb.amazonaws.com` name.
- **Lab variant:** the self-signed shortcut (a leaf that is its own trust anchor, imported with no chain) is the appendix of `greenfield-deployment.md`. Production rotation always uses an enterprise-CA leaf + chain as above.

---

2. Okta client-secret rotation (gateway, Grafana & portal)

Trigger / Frequency: Okta client secret expiry/rotation policy, suspected exposure, or an Okta app re-key. The gateway OIDC secret, the Grafana SSO secret, and (when stack `04` is deployed) the download portal's OIDC secret all ride the same `put_secret_and_roll` helper (hidden prompt → mode-600 `file://` write → forced ECS new-deployment).

Preconditions:

- **Coordinate with the Okta admin first.** Generate the new client secret in Okta and have its value in hand *before* running the script. Okta apps can hold two secrets during an overlap window — ask the admin to add the new secret without removing the old one, so there is no outage between "secret written to Secrets Manager" and "old secret retired in Okta".
- `GATEWAY_STACK_NAME` deployed (gateway secret); `OBS_STACK_NAME` deployed (Grafana secret); `PORTAL_STACK_NAME` deployed (portal secret).

Steps (exact commands):

- **Gateway OIDC secret** — reads `OktaClientSecretArn`, `ClusterName`, `ServiceName` from the gateway stack and rolls the gateway service:

```
bash ./scripts/set-okta-secret.sh # prompts "Okta client secret (input hidden):" — paste the NEW value
```

- **Grafana SSO secret** — reads `GrafanaOidcSecretArn` + `GrafanaServiceName` from the observability stack and `ClusterName` from the gateway stack (shared cluster), and rolls the Grafana service:

```
bash ./scripts/set-grafana-oidc-secret.sh # prompts "Okta client secret for Grafana (input hidden):" — paste the NEW value
```

If Grafana reuses the gateway's Okta app, this is the *same* secret value as `set-okta-secret.sh`.

- **Portal OIDC secret** (when stack `04` is deployed) — reads `PortalOidcSecretArn` + `PortalServiceName` from the portal stack and `ClusterName` from the gateway stack (shared cluster), and rolls the portal service:

```
bash ./scripts/set-portal-oidc-secret.sh # prompts hidden — paste the NEW value
```

If the portal reuses the gateway's Okta app (a documented option — `scripts/deploy.env.example`), this is the *same* secret value as `set-okta-secret.sh`; rotate **both** consumers in the same window.

Verification:

- Watch the roll to stable. The `put_secret_and_roll` helper echoes the exact `aws ecs wait services-stable …` command (with the resolved cluster/service) at the end of its run — copy and run that line.
- Gateway: `scripts/verify-gateway.sh` step 3/3 — the OAuth endpoints respond and issue a device `user_code`.
- Grafana: sign in at `https://${GATEWAY_FQDN}/grafana` via Okta.
- Portal: sign in at `https://${GATEWAY_FQDN}/portal` via Okta and reach the Cost-Center selection page.

Rollback / recovery: If login breaks after the roll, re-run the same script and paste the **previous** secret value (Okta still honours it during the overlap window). Then investigate before retrying.

Failure mode — rolling before Okta has the new value: If you write a new secret to Secrets Manager and roll the service while Okta still expects the old one, the OAuth code exchange fails and logins break for everyone (Grafana with the login form disabled = total lockout). Always confirm the new secret is active in Okta first; keep the old secret valid until the new tasks are stable.

Notes & pitfalls:

- Never pass the secret on a command line — the scripts prompt for it (hidden) and write via a mode-600 `file://` temp file (`.claude/rules/security.md`).
- The template resources are placeholders (`REPLACE-ME-…`). Do **not** "rotate" by editing the template `SecretString` — that clobbers the live value on the next deploy (`.claude/rules/cloudformation.md`).

---

3. Database app-credential rotation

Trigger / Frequency: Normally **automatic** — the db-admin rotation Lambda (`${NAME_PREFIX}-db-rotation`) runs on the `APP_SECRET_ROTATION_DAYS` cadence (`deploy.env`, default 90; passed through as the `AppSecretRotationDays` template parameter). Manual triggers: suspected credential exposure, or recovery after a half-completed rotation.

Design (how it works — describe, don't guess): The secret `${NAME_PREFIX}/db-app-user` alternates between two Postgres LOGIN users, `gateway_app` and `gateway_app_clone`, both of which assume the NOLOGIN owner role `gateway_owner` at login. Each rotation flips `AWSCURRENT` to the *other* user with a fresh password, so the **previous credential stays valid until the next rotation** — there is no window where a running task holds a dead credential. The four standard Secrets Manager steps (`docker/db-admin/app.py:rotate_handler`, semantics fixed by `tests/lambda/test_rotation.py`):

1. `createSecret` — put the pending value (other user + random password) at `AWSPENDING` (idempotent on retry).
2. `setSecret` — `ALTER ROLE <pending user> WITH PASSWORD …` as the RDS master.
3. `testSecret` — connect as the pending user and `SELECT 1`.
4. `finishSecret` — move `AWSCURRENT` to the new version (idempotent), then `forceNewDeployment` on the gateway service so new tasks fetch the new credential.

Preconditions: `DB_STACK_NAME` + `GATEWAY_STACK_NAME` deployed; the db-admin Lambda image is current.

Steps (exact commands):

- **Confirm the last rotation succeeded:**

```bash # Rotation metadata: last-rotated date, schedule, and which versions hold # which stages (expect one AWSCURRENT, and AWSPREVIOUS = the still-valid # prior credential). aws secretsmanager describe-secret --region "$AWS_REGION" \ --secret-id "$NAME_PREFIX/db-app-user" \ --query '{LastRotated:LastRotatedDate, RotationEnabled:RotationEnabled, Stages:VersionIdsToStages}'

# Rotation Lambda log group — look for "rotation finished; service roll requested" aws logs tail "/aws/lambda/$NAME_PREFIX-db-rotation" --region "$AWS_REGION" --since 100d ```

- **Trigger a manual rotation:**

```
bash aws secretsmanager rotate-secret --region "$AWS_REGION" \ --secret-id "$NAME_PREFIX/db-app-user"
```

(Rotation is asynchronous; the CLI returns immediately. Watch the log group above for the four steps.)

*Verification:*

- `describe-secret` shows a newer `LastRotatedDate` and `AWSCURRENT` on a new version id; `AWSPREVIOUS` points at the prior version.
```

- The `${NAME_PREFIX}-db-rotation-errors` alarm stays `OK` (threshold: ≥3 Lambda errors/hour — it tolerates the expected single Inactive-image error per scheduled rotation).
- Gateway tasks are stable after the `finishSecret` roll (`aws ecs wait services-stable … --services $NAME_PREFIX-gateway`).

Rollback / recovery — half-completed rotation: The design is retry-first: Secrets Manager retries a failed step, `finishSecret`'s label move is idempotent, and the prior credential remains valid, so a stuck rotation does **not** break running tasks — they keep working on the current credential.

- If a rotation is wedged, inspect `/aws/lambda/$NAME_PREFIX-db-rotation` for the failing step, fix the cause (common: the image Lambda is `Inactive` and the first invoke fails while Lambda re-optimizes — Secrets Manager's retries then complete it), and re-run `rotate-secret` to re-drive it.
- To abandon an in-flight `AWSPENDING` version without applying it, remove the `AWSPENDING` stage from that version (`update-secret-version-stage --remove-from-version-id …`); the live `AWSCURRENT` credential is untouched.
- **Do not** hand-edit `${NAME_PREFIX}/db-app-user` — it is Lambda-managed (least-privilege AC-6 design). Hand-editing desyncs the secret from the Postgres role passwords.

Notes & pitfalls:

- The gateway **never** uses the RDS master credential; the master secret is break-glass only (see runbook 7). Rotation `ALTER ROLE`s run as master inside the Lambda, not from any task.

---

4. RDS CA bundle refresh

Trigger / Frequency: AWS rotates the RDS server CA (e.g. the `rds-ca-rsa2048-g1` family — the instance's `CACertificateIdentifier`), or you must move to a newer CA before an AWS-announced expiry. Rare (multi-year).

Why this is an image rebuild, not a config flip: both the gateway and the db-admin Lambda connect with `sslmode=verify-full`, and the driver trusts the **OS/container trust store** — it ignores `sslrootcert=`. The RDS CA bundle is staged on the **egress host** by `scripts/mirror/mirror-rds-ca-bundle.sh` (`RDS_CA_BUNDLE_URL`, default the GovCloud truststore) into `mirror/rds-ca-bundle.pem` and **baked into both images** at build (`docker/rds-ca-bundle.pem`, `docker/db-admin/rds-ca-bundle.pem`) — the offline build machine never fetches it (`.claude/rules/offline-build.md`). A CA change therefore means: re-mirror the bundle → transfer → rebuild **both** images with a **bumped immutable tag** → stack update that rolls the services and re-points the Lambda images.

Preconditions: The egress host (reaches the RDS truststore) to refresh the mirrored bundle, the offline build host with Docker and the transferred `mirror/` directory, `KMS_KEY_ARN` set (CMK-encrypted ECR), and — if the CA identifier itself changes on the instance — a maintenance window (modifying `CACertificateIdentifier` on the DB may require a reboot).

Steps (exact commands):

1. **Re-mirror the bundle on the egress host and transfer it:**

```
bash # egress host — override RDS_CA_BUNDLE_URL for commercial regions ./scripts/mirror/mirror-rds-ca-bundle.sh # stages mirror/rds-ca-bundle.pem # copy mirror/ to the build machine (same transfer as a release mirror)
```

1. **Rebuild the gateway image with a bumped tag** (tags are IMMUTABLE — a same-tag rebuild cannot be pushed). The build script re-stages the bundle from the transferred `mirror/` every run:

```
bash # bump the tag so the new bundle ships under a new immutable URI IMAGE_TAG="${CLAUDE_VERSION}-ca$(date +%Y%m%d)" ./scripts/build-and-push-image.sh # persists IMAGE_URI back into deploy.env
```

1. **Rebuild the db-admin Lambda image with a bumped tag:**

```
bash DBADMIN_VERSION="1.0.1" ./scripts/build-and-push-dbadmin.sh # persists DBADMIN_IMAGE back into deploy.env
```

1. **Deploy the gateway stack** so the task definition and both Lambdas pick up the new image URIs and the service rolls (images **before** the stack update that expects them — `.claude/rules/scripts.md`):

```
bash ./scripts/deploy-gateway.sh
```

1. **Only if the instance CA identifier changes:** update `CACertificateIdentifier` on the RDS instance in `01-database.yaml` to the new CA and `./scripts/deploy-database.sh`. This is a property modification, not a replacement — verify it is not flagged as `Update:Replace` before applying (the stack policy denies replacement of `Database`).

Verification:

- Gateway + db-admin connect with `verify-full` against the new CA: `scripts/verify-gateway.sh` passes end to end, and a manual `aws secretsmanager rotate-secret --secret-id "$NAME_PREFIX/db-app-user"` completes its `testSecret` step (proves the rebuilt db-admin image trusts the new CA).
- `aws ecs wait services-stable … --services $NAME_PREFIX-gateway`.

Rollback / recovery: Redeploy with the previous `IMAGE_URI` / `DBADMIN_IMAGE` (both still exist under their old immutable tags in ECR) via `deploy.env` + `deploy-gateway.sh`. Because AWS RDS CA changes are additive (old + new CA trusted during the transition window), the old images keep validating until the old CA is retired.

Notes & pitfalls:

- Rebuild **both** images — a refreshed gateway with a stale db-admin image (or vice versa) leaves one side unable to validate `verify-full` after the CA fully cuts over.
- Do not attempt to fix a CA change by setting `sslrootcert=` — the driver ignores it. The only lever is the baked-in bundle.

---

5. Claude Code release update

Trigger / Frequency: A new pinned Claude Code release you want to distribute (security fix, feature, or a gateway-required minimum bump).

Preconditions:

- An egress host that can reach `downloads.claude.ai` (the mirror step). The laptops and the container build need **no** egress.
- `ANTHROPIC_GPG_KEY` set to Anthropic's release-signing public key so the manifest signature is verified. Verification **fails closed**: `ALLOW_UNVERIFIED_MANIFEST=1` is the only (deliberate, named) escape hatch and must not be the default (`.claude/rules/security.md`).

Steps (exact commands):

1. **Mirror the release** (linux-x64 for the image *and* the portal's Linux download; win32-x64 for Windows laptops):

```
bash ANTHROPIC_GPG_KEY=/path/to/anthropic-release-key.asc \ ./scripts/mirror/mirror-claude-release.sh 2.1.208 # verifies the GPG-signed manifest + per-binary SHA-256, writes # mirror/2.1.208/{claude,claude.exe,CHECKSUMS.txt}
```

Then copy `mirror/` to the build machine — it builds offline (`.claude/rules/offline-build.md`) and also expects the previously transferred `mirror/rds-ca-bundle.pem` to still be present (the gateway build bakes it in every run).

1. **Rebuild the gateway image** (it embeds the linux binary). Set `CLAUDE_VERSION` to the new release in `deploy.env` first; the build script stages the binary from `mirror/<version>/` itself, re-verifying it against the mirror's `CHECKSUMS.txt`:

```
bash # deploy.env: export CLAUDE_VERSION="2.1.208" ./scripts/build-and-push-image.sh # tags the image 2.1.208, persists IMAGE_URI
```

Do **not** run `deploy-gateway.sh` yet when stack `04` is deployed: the deploy raises the pushed `requiredMinimumVersion` client floor to the new `CLAUDE_VERSION` (§6 note), and update-locked clients can only comply through the portal — publish there first (step 3), then roll (step 4). To roll the gateway without raising the floor, pin `MIN_CLIENT_VERSION` in `deploy.env` to the old version for the run.

1. **Publish to the download portal** (when stack `04` is deployed) — this is how developers self-serve the new version. Reuses the verified mirror output; uploads both platform binaries (`claude.exe` + `claude`), `manifest.json`, `CHECKSUMS.txt`, both installers (`Install-ClaudeCode.ps1` + `install-claude-code.sh`), and the user-guide PDF (`docs/generated/user-manual.pdf` → the bucket's `docs/user-manual.pdf`, served at `/portal/guide`; the script fails fast if the PDF is missing — `make docs-pdf` builds it, `SKIP_USER_GUIDE=1` is the named skip) to the portal's CMK-encrypted artifacts bucket, then pins the portal to the new version:

```
bash ./scripts/publish-portal-release.sh 2.1.208 # deploy.env: export PORTAL_RELEASE_VERSION="2.1.208" (empty = CLAUDE_VERSION) ./scripts/deploy-download-portal.sh # only needed when the pinned version changes
```

1. **Roll the gateway service** onto the new image — this is also the moment the pushed client floor rises to the new version (unless pinned):

```
bash ./scripts/deploy-gateway.sh # rolls the service; pushes requiredMinimumVersion=<CLAUDE_VERSION>
```

1. **Distribute the Windows client (share/MDM route).** Stage `mirror/2.1.208/claude.exe` + `CHECKSUMS.txt` on the file share and install non-elevated per developer:

```
powershell .\client\Install-ClaudeCode.ps1 ` -BinaryPath \\fileserver\software\claude\2.1.208\claude.exe ` -Sha256 <win32-x64 checksum from CHECKSUMS.txt> ` -GatewayUrl https://<GATEWAY_FQDN> -DisableUpdates
```

1. **Forcing the upgrade — automatic.** `deploy-gateway.sh` (step 4) defaults the pushed `requiredMinimumVersion` floor to the new `CLAUDE_VERSION`, so every client that has fetched `/managed/settings` refuses to start on the old version at its next launch, with instructions to update (see `client-config.md` §6e). No GPO change is needed. Pin `MIN_CLIENT_VERSION` in `deploy.env` (or set it `none`) to hold the floor while the fleet catches up. A GPO/MDM-delivered `requiredMinimumVersion` (`client-config.md` §8) still works as a machine-policy copy, but is no longer the mechanism of record — and don't pin a *different* value there: which managed source wins when both set a floor is not verified anywhere, so keep the GPO JSON free of the key unless you deliberately manage the floor there instead.

Verification:

- Mirror step: the script prints `checksum OK` per platform and `manifest signature OK`. A SHA-256 or signature mismatch aborts (non-zero exit) and removes the bad file.
- Gateway: `aws ecs wait services-stable … --services $NAME_PREFIX-gateway`, then `scripts/verify-gateway.sh`.
- Client: `claude --version` reports the new version; below-floor binaries refuse to start when `requiredMinimumVersion` is raised.
- Portal (if published): download a ZIP per platform from `https://${GATEWAY_FQDN}/portal` and confirm the Windows one contains the new `claude.exe` (generated `install.cmd` carrying the new win32-x64 SHA-256) and the Linux one the new `claude` (generated `install.sh` carrying the linux-x64 SHA-256); each download appears in the portal audit log group (`/claude/${NAME_PREFIX}/portal-audit`) with its `platform` field.

Rollback / recovery: Redeploy the previous `IMAGE_URI` (old immutable tag still in ECR) via `deploy.env` + `deploy-gateway.sh` — and **revert** `CLAUDE_VERSION` **(or pin** `MIN_CLIENT_VERSION`**) in the same edit**: the pushed client floor defaults from `CLAUDE_VERSION`, so restoring only `IMAGE_URI` leaves the *next* `deploy-gateway.sh` pushing the NEW floor while the portal and fleet are back on the old version — locking out every client that hasn't updated. If you had raised a GPO-delivered floor instead, lower it there (`client-config.md` §8). Portal: set `PORTAL_RELEASE_VERSION` back to the prior version and re-run `deploy-download-portal.sh` (earlier `releases/<version>/` prefixes stay in the artifacts bucket). Keep the prior `mirror/<version>/` directory until the new release is confirmed across the fleet.

Notes & pitfalls:

- Update lockdown (`-DisableUpdates` → `DISABLE_UPDATES=1` + `DISABLE_AUTOUPDATER=1`) is what keeps users on the distributed version — do not drop it, or clients will self-update off the pinned build.
- **The installer is user-scope only** — it must run in the developer's own (non-elevated) context. A SYSTEM-context run is **refused outright** (it would install the binary into SYSTEM's profile and PATH, which no developer sees), and there is no settings-push mode: the installer writes only the user settings `env`

block (`DISABLE_UPDATES` / `DISABLE_AUTOUPDATER` , `OTEL_RESOURCE_ATTRIBUTES` , `NODE_EXTRA_CA_CERTS`), never a machine/policy source. For a device-context binary push use the MDM "user" install behavior (Intune) or the download portal.

- **Gateway login REQUIRES the managed setting — an admin-channel concern, not the installer's.** `forceLoginMethod: "gateway"` + `forceLoginGatewayUrl` are what make the **Cloud gateway** login path appear at all; without them `/login` shows only the standard account picker with **no gateway option and no way for a user to type a gateway URL** (Anthropic's anti-phishing design). These keys are honored **only** from a **managed source** — `HKLM\SOFTWARE\Policies\ClaudeCode` or `%ProgramFiles%\ClaudeCode\managed-settings.json` — **never** from user `settings.json` or HKCU (a user-level `forceLoginMethod:"gateway"` is explicitly nulled by the binary), which is why the installer writes no policy source. (`requiredMinimumVersion` *can* ride the same managed JSON, but the gateway pushes the floor itself — see step 6 above — so keep the key out of the GPO copy unless deliberately managing it there.) On hardened fleets the `Policies` subtree is ACL-locked under STIG/CIS baselines, so the entry is delivered by GPO/MDM: a GPP Registry `REG_SZ` value `Settings` under `HKLM\SOFTWARE\Policies\ClaudeCode` , or a GPP Files copy of `managed-settings.json` to `%ProgramFiles%\ClaudeCode\` (Claude Code moved off `%ProgramData%` at v2.1.75; admin-write-only = tamper-resistant). A developer **with local admin** can self-serve the entry once. Full AD-admin steps are in `client-config.md` §8; the AD/GPO request template is `ad-request-email.md` . The **binary install stays no-admin either way** — only the login config needs the managed setting.
- Never bypass GPG verification as a matter of routine; `ALLOW_UNVERIFIED_MANIFEST=1` is for a deliberately air-gapped one-off only.

---

6. Gateway / Grafana / collector image & stack updates

Trigger / Frequency: Any container change (Dockerfile fix, Grafana provisioning, a new ADOT collector release, task CPU/memory tuning, or a parameter change). Note the collector is a **sidecar in the gateway task**, not a standalone service: rolling it is a new **gateway task-definition revision**, done by re-running `deploy-gateway.sh` , not `deploy-observability.sh` .

Model changes (`OPUS_MODEL_ID` / `SONNET_MODEL_ID` / `HAIKU_MODEL_ID` *and their* `*_BEDROCK_MODEL_ID` *pairs) are a special case:* these parameters drive three things at once — the Bedrock routing, the scoped IAM / endpoint policies, and the **client model allowlist the gateway pushes via** `/managed/settings` . Changing them therefore changes what every developer's `/model` picker offers; the pushed `ANTHROPIC_DEFAULT_HAIKU_MODEL` (background/small-model) override follows `HAIKU_MODEL_ID` . The three gateway-facing IDs must be distinct — `deploy-gateway.sh` fails fatal on duplicates before deploying (the usual hit: a pre-Sonnet-5 `deploy.env` whose `SONNET_MODEL_ID` is still `claude-sonnet-4-5` , colliding with the `HAIKU_MODEL_ID` default). Re-run `deploy-gateway.sh` ; connected clients pick the new allowlist up on their next settings fetch (re-login if it lags). Verify with `/model` on a real client before calling it done.

Gateway version bumps (`CLAUDE_VERSION` *) also raise the client version floor:* `deploy-gateway.sh` defaults the `MinClientVersion` parameter to `CLAUDE_VERSION` , and the gateway pushes it to every client as `requiredMinimumVersion` via `/managed/settings` — a client older than the floor exits at its next start (after it has fetched settings) with instructions to update. Auto-updates are locked down, so **publish the matching installer to the portal (** `publish-portal-release.sh` **) before or with the** `deploy-gateway.sh` **re-run**, or users hit the startup error with no way to comply. Pin `MIN_CLIENT_VERSION` in `deploy.env` to hold the floor while rolling the gateway ahead, or set it to `none` to disable the check. See `client-config.md` §6e.

Preconditions: Build host with Docker; `KMS_KEY_ARN` set; the relevant stack already deployed. **Rebuild and push the image BEFORE the stack update that expects it** (`.claude/rules/scripts.md`).

Steps (exact commands): bump the immutable tag → build/push → deploy.

- **Gateway:** `IMAGE_TAG=<new> ./scripts/build-and-push-image.sh` → `./scripts/deploy-gateway.sh`.
- **Base images (all four):** `./scripts/mirror/mirror-base-images.sh [gateway|lambda|grafana|portal ...]` on the dual-reach host (upstream registries + AWS creds) re-mirrors and re-pins the `*_BASE_IMAGE` vars → copy the updated lines into the build machine's `deploy.env` if the hosts differ → rebuild the consuming image(s) with bumped tags → deploy. A digest-pinned `*_BASE_IMAGE` in `deploy.env` **always wins over the build script's version-derived default** — see the Grafana note below.
- **Grafana:** `GRAFANA_IMAGE_TAG=<new> ./scripts/build-and-push-grafana.sh` → `./scripts/deploy-observability.sh`. **Version bumps in the target profile start with the base re-mirror:** once `mirror-base-images.sh` has pinned `GRAFANA_BASE_IMAGE` in `deploy.env`, the build ignores `GRAFANA_VERSION` for the base — bumping only the tag would rebuild the OLD Grafana under a NEW name. Run `GRAFANA_VERSION=<new> ./scripts/mirror/mirror-base-images.sh grafana` first (and carry the updated `GRAFANA_BASE_IMAGE` line over), then build. Notes for version bumps: the OSS image is `grafana/grafana` (the `grafana/grafana-oss` Docker Hub repo froze at 12.4); the build script also stages the `grafana-amazonprometheus-datasource` plugin into the image (SigV4 left the core prometheus datasource in 13.1 and the task has no egress to install plugins at boot) — the plugin zip must already sit in the transferred `mirror/grafana-plugins/` (run `scripts/mirror/mirror-grafana-plugin.sh` on the **egress host**, then copy `mirror/` over; the build host never fetches — `.claude/rules/offline-build.md`); the pin lives in `scripts/mirror/grafana-plugin.pin` — bump `AMP_PLUGIN_VERSION` + `AMP_PLUGIN_SHA256` together there when updating it, then re-mirror and re-transfer; and expect a **one-time re-login for all Grafana users** on the first post-upgrade start (external OAuth sessions are re-linked — `improvedExternalSessionHandling`, default-on since 12.1).
- **ADOT collector (sidecar):** `ADOT_VERSION=<vX.Y.Z> ./scripts/mirror/mirror-collector.sh` (mirrors + pins `COLLECTOR_IMAGE` by digest) → `./scripts/deploy-gateway.sh` (the sidecar lives in the gateway task, so the collector rolls with a new gateway task-def revision — **not** an observability-stack update).
- **db-admin Lambda:** `DBADMIN_VERSION=<new> ./scripts/build-and-push-dbadmin.sh` → `./scripts/deploy-gateway.sh`.
- **Download portal** (a Flask/gunicorn package with committed vendored wheels):
- *Only when* `docker/portal/requirements.txt` *changed:* on the **egress host**, `./scripts/mirror/mirror-python-deps.sh portal` regenerates the committed `docker/portal/vendor/` wheel set — commit the wheel changes (the build host installs `--no-index` from `vendor/` and never fetches; `.claude/rules/offline-build.md`).
- `PORTAL_VERSION=<new> ./scripts/build-and-push-portal.sh` (immutable tags — always bump).
- *Only when upgrading a portal deployment older than the spend-read-key import:* re-run `./scripts/deploy-gateway.sh` **first** — 04 imports 02's `${NAME_PREFIX}-spend-read-key-arn` export (the read-only spend key that powers `/portal/me`), and the 04 deploy fails on the missing import against an older 02.
- `./scripts/deploy-download-portal.sh`.
- `./scripts/publish-portal-release.sh <version>` if the release pin or the user guide changed (it uploads the guide PDF too — runbook 5 step 3).

Checks after a portal roll: sign in at `https://${GATEWAY_FQDN}/portal`; download a ZIP end to end at real size and confirm the download lands in the portal audit log group; `/portal/me` shows the signed-in user's caps + period-to-date spend; `/portal/admin/users` lists, searches, and pages (as a `PORTAL_ADMIN_GROUP` member); `/portal/fingerprint` matches the served-cert SHA-256 that `scripts/verify-gateway.sh` reports; `/portal/guide` renders and its download link returns the full PDF.

Each build script persists its new URI/tag into `deploy.env`, so the matching `deploy-*.sh` picks it up with no copy-paste.

Update-safety invariants (must hold on every stack update — `.claude/rules/cloudformation.md`):

- **The ALB and RDS instance must never be replaced by a routine update.** Both are protected three ways — deletion protection, fixed physical names, and a **stack policy** (set by `deploy-gateway.sh` / `deploy-database.sh`) denying `Update:Replace`/`Update:Delete` on `LoadBalancer` / `Database`. Do not remove any layer. `deploy-gateway.sh` deploys with `--disable-rollback` by default (`CFN_DISABLE_ROLLBACK=true`) precisely so a failed create keeps the protected ALB rather than attempting an impossible rollback delete.
- **Cross-stack exports are locked while imported.** 01 exports the CMK, DB endpoint, master-secret ARN, and client SG to 02; 02 exports SGs, the listener, and the cluster to 03. You cannot change an exported value in place while a downstream stack imports it — encryption-at-rest and resource names are day-one decisions.
- **Placeholder `SecretString` resources must not be touched.** `OktaClientSecret`, `GrafanaOidcClientSecret`, `PortalOidcClientSecret`, and `DbAppUserSecret` hold placeholder/managed values; editing the `SecretString` literal (or the resource Name/Description) re-applies the placeholder and clobbers the live secret. Rotate via the scripts (runbooks 2–3), never the template.
- `TaskCpu`/`TaskMemory` **must stay a valid Fargate pairing** — the template's `Rules` section asserts this; an invalid combo fails deploy with an opaque error. Change them via `TASK_CPU`/`TASK_MEMORY` in `deploy.env`.

Verification: `aws ecs wait services-stable` for the affected service (`$NAME_PREFIX-gateway`, `$NAME_PREFIX-grafana`, or `$NAME_PREFIX-portal` — there is no separate collector service; a collector roll is a `-gateway` roll); `scripts/verify-gateway.sh` for the gateway; Grafana login + dashboards for Grafana. For a collector-sidecar change, confirm the gateway task runs both containers (`gateway` + `otel-collector`) and metrics still reach AMP. Confirm the CloudFormation events show `UPDATE_COMPLETE` with **no** replacement of `LoadBalancer` or `Database`.

Rollback / recovery: Redeploy the prior image URI/tag from `deploy.env` (old immutable tags persist in ECR). For a parameter regression, re-run the deploy script with the previous `deploy.env` values. If a gateway create/update lands in `*_FAILED` with `--disable-rollback`, fix the cause and re-run `deploy-gateway.sh` — the deploy **continues** from where it failed rather than tearing down the protected ALB.

Notes & pitfalls: A same-tag rebuild cannot be pushed (immutable repos) and an unchanged image URI leaves the service/Lambda on old code — always bump the tag.

---

7. Secrets inventory & break-glass

Trigger / Frequency: Reference during audits, incident response, or before any secret change.

Secrets inventory (all CMK-encrypted with the CMK from 01):

Secret (Name)	Stack	Rotation	How to rotate
RDS master `${NAME_PREFIX}` DB (`Database.MasterUserSecret`)	01	**Automatic**, RDS-managed, every 7 days	RDS-managed; break-glass use → force-rotate (below)
`${NAME_PREFIX}/db-app-user`	02	**Automatic**, alternating-user Lambda, `AppSecretRotationDays` (default 90)	Runbook 3
`${NAME_PREFIX}/oidc-client-secret` (Okta)	02	**Manual**	`scripts/set-okta-secret.sh` (runbook 2)
`${NAME_PREFIX}/jwt-secret` (session signing)	02	**Not rotated automatically** (`GenerateSecretString` at create)	Manual — see below
`${NAME_PREFIX}/grafana-oidc-client-secret`	03	**Manual**	`scripts/set-grafana-oidc-secret.sh` (runbook 2)
`${NAME_PREFIX}/grafana-admin-password`	03	**Not rotated** (break-glass; login form disabled)	Regenerate manually (below)
`${NAME_PREFIX}/spend-admin-write-key`	02	**Not rotated automatically** (`GenerateSecretString` at create)	Manual — procedure in `cost-controls.md` §7 "Key rotation" (file-based write, then force a gateway deployment)
`${NAME_PREFIX}/spend-admin-read-key`	02	**Not rotated automatically** (`GenerateSecretString` at create)	Same as the write key (`cost-controls.md` §7) — **the portal task injects it too** (`SPEND_READ_KEY`, via the 02 export), so after writing a new value force-roll `$NAME_PREFIX-portal` as well as `$NAME_PREFIX-gateway`
`${NAME_PREFIX}/portal-oidc-client-secret`	04	**Manual**	`scripts/set-portal-oidc-secret.sh` (runbook 2)
`${NAME_PREFIX}/portal-session-secret` (cookie signing)	04	**Not rotated automatically** (`GenerateSecretString` at create)	Same file-based pattern as the JWT secret (below), then roll `$NAME_PREFIX-portal`; rotation invalidates portal sessions (users just re-login)

Gateway JWT secret (manual rotation). The template describes rotation as "prepend new value, roll, remove old." Whether the gateway honours two overlapping signing keys has not been exercised here, so plan a JWT rotation as **session-invalidating**: active sessions below `SessionTtlHours` (default 1 h) are dropped and users re-login. Rotate by writing a new value via the same safe pattern the helper uses, then forcing a roll:

```
JWT_ARN=$(aws cloudformation describe-stack-resources --region "$AWS_REGION" \
  --stack-name "$GATEWAY_STACK_NAME" --logical-resource-id JwtSecret \
  --query 'StackResources[0].PhysicalResourceId' --output text)
# generate + write without ever putting the value on argv:
f=$(mktemp); chmod 600 "$f"; aws secretsmanager get-random-password \
  --region "$AWS_REGION" --password-length 48 --exclude-punctuation \
  --query RandomPassword --output text > "$f"
aws secretsmanager put-secret-value --region "$AWS_REGION" \
  --secret-id "$JWT_ARN" --secret-string "file://$f"; rm -f "$f"
```

```
aws ecs update-service --region "$AWS_REGION" \
  --cluster "$(aws cloudformation describe-stacks --region "$AWS_REGION" \
    --stack-name "$GATEWAY_STACK_NAME" \
    --query "Stacks[0].Outputs[?OutputKey=='ClusterName'].OutputValue" --output text)" \
  --service "$NAME_PREFIX-gateway" --force-new-deployment
```

Grafana admin password (break-glass regenerate). Same file-based pattern against `${NAME_PREFIX}/grafana-admin-password`, then roll `$NAME_PREFIX-grafana`. The login form stays disabled unless `GRAFANA_DISABLE_LOGIN_FORM=false`; day-to-day access is Okta SSO.

Break-glass — RDS master secret. The master credential is **break-glass ONLY**; no task ever injects it (the gateway uses `${NAME_PREFIX}/db-app-user`). Use it only for direct DBA access during an incident.

Steps:

1. Read it without echoing to the terminal history:

`bash MASTER_ARN=$(aws cloudformation describe-stacks --region "$AWS_REGION" \ --stack-name "$DB_STACK_NAME" \ --query "Stacks[0].Outputs[?OutputKey=='DBMasterSecretArn'].OutputValue" --output text) # inspect keys interactively; avoid persisting the value anywhere aws secretsmanager get-secret-value --region "$AWS_REGION" \ --secret-id "$MASTER_ARN" --query SecretString --output text`

1. Perform the minimum necessary DBA action, from a host that can reach the DB (in-VPC operator host; the DB is `PubliclyAccessible: false`).
2. **Immediately afterwards — rotate the master back** so the just-exposed value is retired, and **record the use** (who/when/why) for the audit trail:

`bash aws secretsmanager rotate-secret --region "$AWS_REGION" --secret-id "$MASTER_ARN"`

Verification: `describe-secret` on the master shows a fresh `LastRotatedDate` after the forced rotation; the gateway is unaffected (it never used the master).

Notes & pitfalls: Treat the activity-log stream as highly sensitive (bash commands, tool inputs, file paths per user) — opt-in, IAM-only, CMK-encrypted, SIEM-flagged; never widen its access surface. Never put any secret value on a command line (`--secret-string <value>` leaks via `ps` / `/proc`); always the mode-600 `file://` pattern above.

---

8. Backup & restore

Trigger / Frequency: Reference for DR planning; act on data-loss/corruption or before a risky change.

Posture:

- **RDS automated backups** — `BackupRetentionPeriod = BackupRetentionDays` (`01-database.yaml`, default 14, max 35). Daily automated snapshots + PITR within the window. `DeletionPolicy: Snapshot` / `UpdateReplacePolicy: Snapshot` → a stack delete/replace takes a **final snapshot** rather than destroying data. `DeletionProtection: true` blocks accidental instance deletion.
- **ALB access logs** — `AlbLogsBucket` (SSE-S3; ELB delivery does not support KMS — this is the one documented CMK exception), `DeletionPolicy: Retain`, lifecycle expiry at `AlbLogRetentionDays` (default 90).
- **Activity archive** — `ActivityArchiveBucket` (CMK-encrypted, `DeletionPolicy: Retain`), lifecycle expiry at `ActivityArchiveRetentionDays` (default 731 ≈ 2y). The CloudWatch window group `/claude/${NAME_PREFIX}/activity` retains `ActivityLogWindowDays` (default 14) before the Firehose→S3 chain is the durable copy. Idle (no cost) until the gateway sets `ForwardActivityLogs=true`.
- **AMP** — the workspace is `DeletionPolicy: Retain` so a routine 03 recreate does not destroy metrics history.

Take an on-demand snapshot (before risky changes):

```
aws rds create-db-snapshot --region "$AWS_REGION" \
  --db-instance-identifier "$NAME_PREFIX-store" \
  --db-snapshot-identifier "$NAME_PREFIX-store-preop-$(date +%Y%m%d%H%M)"
```

Restore — understand the blast radius first. **A replaced RDS instance is an EMPTY database, not a restore** (`.claude/rules/cloudformation.md`), and the DB endpoint is a cross-stack export imported by 02, which is **locked while imported**. You cannot restore in place by pointing the stack at a snapshot without disturbing that export. Restore is therefore effectively a **teardown + restore**, not an update:

1. Restore the snapshot to a **new** instance out-of-band to validate the data (`aws rds restore-db-instance-from-db-snapshot …`), or use PITR.
2. To make the restored data the live store, the sanctioned path is to bring the database stack back from the snapshot (RDS snapshot-based restore) with the same `DBInstanceIdentifier`/exports so 02 re-imports the endpoint — which, given the export lock and deletion protection, means an orchestrated teardown of 02 first, restore of 01 from the snapshot, then redeploy of 02/03. Plan this as a maintenance-window operation, not a routine update.

Verification: `aws rds describe-db-snapshots` shows the expected automated + manual snapshots; a test restore to a scratch instance connects and shows the expected schema/rows. After a real restore, `scripts/verify-gateway.sh` passes and the gateway serves logins.

Rollback / recovery: Snapshots are immutable point-in-time copies — a failed restore attempt is retried against another snapshot; the source snapshots are unaffected. Keep the pre-op on-demand snapshot until the operation is confirmed.

Notes & pitfalls: Per the deferred-Object-Lock decision, neither S3 bucket uses Object Lock — archived logs are deletable by a privileged operator. If tamper-evidence becomes a requirement, revisit that decision (`../ato/security-assessment-2026-07.md`).

---

9. Alarm response

Trigger / Frequency: On alarm (routed to `ALARM_SNS_TOPIC_ARN` when set — otherwise the alarms exist but have no action).

Alarms defined in the templates:

1. `${NAME_PREFIX}-certificate-expiry` (`02-gateway.yaml`) — `AWS/CertificateManager` `DaysToExpiry` ≤ `CERT_EXPIRY_ALARM_DAYS` (default 30). *Response:* the imported cert is approaching expiry and will **not** auto-renew → execute **runbook 1** (re-issue from the enterprise CA, publish the new fingerprint, in-place `import --certificate-arn`). Alarm clears once ACM recomputes `DaysToExpiry` (daily).

2. `${NAME_PREFIX}-db-rotation-errors` (`02-gateway.yaml`) — `AWS/Lambda` `Errors` ≥ 3 in one hour on `${NAME_PREFIX}-db-rotation`. *Response:* rotation is erroring repeatedly (running tasks are still fine on the current credential, but the rotation SLA is at risk). Inspect `/aws/lambda/$NAME_PREFIX-db-rotation`, identify the failing step, and follow **runbook 3** recovery. The threshold intentionally tolerates the single expected Inactive-image error per scheduled rotation.

3. `${NAME_PREFIX}-missing-telemetry` (`03-observability.yaml`) — no samples ingested into the AMP workspace for `MISSING_TELEMETRY_ALARM_MINUTES` (default 15) consecutive minutes (`AWS/Usage` `ResourceCount` / `Resource=IngestionRate` scoped to the workspace; missing data = breaching, because AMP stops emitting the metric when nothing arrives). This is the **end-to-end backstop for the fail-closed telemetry posture** — container health only proves the collector is alive; this proves data is landing. *Response — triage in this order:* (1) is the gateway service running at all? (a full outage also silences telemetry — check the service first, this alarm may be a symptom); (2) `otel/`-prefixed log streams in the gateway log group for collector/export errors; (3) collector container health in the task detail (fail-closed replaces the task; fail-open leaves it running UNHEALTHY); (4) task-role `aps:RemoteWrite` + `aps-workspaces` endpoint reachability; (5) AMP-side rejection — check `AWS/Prometheus` `DiscardedSamples` for the workspace (throttling/validation). *Expected (not actionable) firings:* between the 03 deploy and the telemetry-enabled 02 re-run, and during deliberate gateway downtime.

4. `${NAME_PREFIX}-missing-activity-logs` (`03-observability.yaml`, **off by default**) — only created when `ACTIVITY_LOGS_ALARM_MINUTES` > 0. No events delivered to the activity audit log group for that many minutes (`AWS/Logs` `IncomingLogEvents`, missing = breaching). This is the audit-stream (AU-2/AU-12) counterpart to alarm 3 — alarm 3 watches *metrics*, this watches the *audit logs* pipeline, which alarm 3 and the collector health check cannot see. *Response:* same triage as alarm 3, plus check the collector's `awscloudwatchlogs` exporter for errors in the `otel/` streams. **Before declaring a fault, confirm the fleet was actually active** — this stream is intermittent, so genuine idleness also reads as silence; correlate with the usage metrics (a live metrics stream

5. dead audit stream = real fault; both quiet = probably just idle). Left off by default for exactly this reason; enable only on continuously-active fleets with a window longer than the longest expected quiet gap.

No other CloudWatch alarms are defined in the templates. Operational surfaces to watch manually: ECS service events / `services-stable`, and the gateway (which now also carries the collector sidecar's `otel`-prefixed streams) and Grafana log groups.

General verification after responding: confirm the alarm returns to `OK` (`aws cloudwatch describe-alarms --alarm-names <name> --query 'MetricAlarms[0].StateValue'`).

Known landing-zone gotcha — ALB access-log AccessDenied. If ALB access-log enablement fails `AccessDenied` on a bucket policy that is correct, **suspect a landing-zone auto-remediation rewriting the ALB's log config before suspecting the bucket policy**. The bucket policy already grants both ELB delivery principals. Get the auto-remediation exempted for this ALB rather than re-editing the policy.

Rollback / recovery: Alarm response is corrective, not stateful — there is nothing to roll back beyond the underlying runbook's own recovery.

9a. Diagnostics — which tool answers which question

Three read-only tools cover the usage/telemetry chain. Symptom-indexed diagnosis (what each failure looks like and how to tell them apart) is `troubleshooting.md`; this is the index of what to reach for.

Tool	Answers
`scripts/diagnostics/diagnose-telemetry.sh`	*Why are the Grafana panels empty?* Reads the ALB access logs (client → `/managed/settings`, client → `/v1/metrics`) and then queries AMP over SigV4, so it separates enrollment, export, remote-write, translation and dashboard-filter failures — which all look identical in Grafana. It also prints the labels actually present on `claude_code_cost_usage`, because absent `team` / `cost_center` / `user_groups` labels render empty panels over present data.
`scripts/diagnostics/amp-query.py`	*What is actually in AMP?* SigV4-signed report of stored `claude_code_*` metric names, the `otelcol_*` heartbeat series, and the collector's own pipeline counters (accepted vs refused vs **failed translations**). Env-driven: `OBSERVABILITY_AMP_ENDPOINT`, `AWS_REGION`, `AMP_QUERY_WINDOW_HOURS` (default 48 — client metrics are bursty and short windows miss them).
`scripts/diagnostics/dump-usage.sh`	*What has the gateway persisted?* Read-only dump of `spend`, `spend_limits`, `principal_emails` and `admin_audit` over the same connection path the gateway uses (app-user secret + RDS CA, verify-full). Postgres holds **aggregate cents per principal per period**, never per-request token counts — those live only in AMP.

`dump-usage.sh` needs two things that are easy to miss, both by design and neither requiring a template change:

1. **Network.** The DB admits only members of the `<prefix>-db-client-sg` security group (stack 01 output `DBClientSecurityGroupId`). Attach that SG to your in-VPC admin instance's ENI — it is additive, SG rules union. Security groups do not apply from outside the VPC, so an off-VPC host needs a bastion. Do **not** widen the DB SG ingress; use the membership SG.
2. **Secret decrypt.** The app-user secret is CMK-encrypted, so the operator role needs both `secretsmanager:GetSecretValue` on `<prefix>/db-app-user` **and** `kms:Decrypt` on the CMK (scoped `kms:ViaService=secretsmanager.<region>.amazonaws.com`). The `kms:Decrypt` half is the non-obvious one.

Dependencies are `botocore` (ships with the AWS CLI) and `pg8000`; on an offline host install the latter from the repo's vendored wheels: `pip install --no-index --find-links docker/db-admin/vendor pg8000`. No `boto3` needed. The tool uses the **app-user** secret, never the RDS master.

---

10. Spend caps (per-user / per-group cost limits)

Moved: spend management has its own runbook — `cost-controls.md`. It covers the enforcement model (the `admin:` master switch; caps as `spend_limits` rows), setting caps via the portal admin page or the break-glass `scripts/set-spend-limit.sh` CLI, monitoring spend (Grafana dashboard, AMP queries, Postgres ground truth), what a capped developer sees, the **fail-closed spend-store outage** incident runbook (a spend-store problem halts all inference fleet-wide — a deliberate availability trade), the audit trail, and known gaps. This section number is kept so existing references to "om-runbooks §10" keep resolving.

---

11. Bedrock prompt logging (model invocation logging)

Trigger / Frequency: Enabling or disabling the capture of verbatim prompts and model responses; access requests to the captured data.

Model: Bedrock's **model invocation logging** is an **account+region-level Bedrock setting**, not a stack resource. Stack 03 **always** creates the destinations (inert, near-free while unused): a CMK CloudWatch group `/claude/<prefix>/bedrock-prompts` (short window, `BEDROCK_PROMPT_LOG_WINDOW_DAYS`) and a CMK S3 bucket (its own full copy — **and the only place request/response bodies over 100 KB ever land**; `BEDROCK_PROMPT_ARCHIVE_RETENTION_DAYS`). Only the account setting toggles: `deploy-observability.sh` applies it from `BEDROCK_PROMPT_LOGGING` (tri-state; empty = never touched; `false` is a no-op when logging is already off, so it is safe to leave standing in deploy.env).

> **Know what you are switching on.** (1) It captures **every Bedrock invocation in the account+region**, not only this gateway's — appropriate in a dedicated workload account, wrong in a shared one. (2) It does **not** attribute prompts to developers: the log identity is the gateway task role. Who-did-what remains the activity stream (runbook 9's pipeline); this adds what-was-said. (3) Sensitivity is strictly above the activity stream — treat access like the activity archive: IAM-only, SIEM-flagged, no ad-hoc grants.

KMS prerequisite (do this first — the enable fails without it): Bedrock delivers to the SSE-KMS bucket only if the CMK's **key policy** grants `bedrock.amazonaws.com` `kms:GenerateDataKey` (statement `BedrockInvocationLogsWrite` in `01-database.yaml`). When it is missing, the enable call fails at the very end with the **misleading** `Failed to validate permissions for bucket ... verify the S3 bucket policy` (the denial is at KMS, not S3; `deploy-observability.sh` preflights this and fails fast with the real cause). How the statement gets onto the key depends on who manages it:

- **Stack-managed key** (01 created it and still owns it — `KmsKeyArn` parameter on the deployed stack is empty): re-run `scripts/deploy-database.sh`; the key-policy update is in-place.
- **Bring-your-own or stack-detached key** (the stack's `KmsKeyArn` parameter holds an ARN — including deployments bitten by the parameter-feedback detach, see §11a): apply the statement out-of-band. Get → append → put preserves every existing statement:

```
ACCT="$(aws sts get-caller-identity --query Account --output text)"
PARTITION="$(cut -d: -f2 <<<"$KMS_KEY_ARN")"   # aws | aws-us-gov
TMP="$(mktemp)"
aws kms get-key-policy --key-id "$KMS_KEY_ARN" --policy-name default \
  --region "$AWS_REGION" --query Policy --output text > "$TMP"
```

```
python3 - "$TMP" "$ACCT" "$AWS_REGION" "$PARTITION" <<'EOF'
import json, sys
path, acct, region, partition = sys.argv[1:5]
p = json.load(open(path))
if any(s.get("Sid") == "BedrockInvocationLogsWrite" for s in p["Statement"]):
    sys.exit("already present - nothing to do")
p["Statement"].append({
  "Sid": "BedrockInvocationLogsWrite", "Effect": "Allow",
  "Principal": {"Service": "bedrock.amazonaws.com"},
  "Action": "kms:GenerateDataKey", "Resource": "*",
  "Condition": {"StringEquals": {"aws:SourceAccount": acct},
                "ArnLike": {"aws:SourceArn": f"arn:{partition}:bedrock:{region}:{acct}:*"}}})
json.dump(p, open(path, "w"), indent=2)
EOF
aws kms put-key-policy --key-id "$KMS_KEY_ARN" --policy-name default \
  --policy "file://$TMP" --region "$AWS_REGION"
rm -f "$TMP"
```

If the key is centrally managed by the landing zone and the deploy role lacks `kms:PutKeyPolicy`, this becomes a request to the key admins — send them the statement (exact shape: `BedrockInvocationLogsWrite` in `01-database.yaml`, partition included).

Enable:

```
# 1. KMS prerequisite above (stack-managed key: scripts/deploy-database.sh;
#    BYO/detached key: the out-of-band statement).
# 2. Destinations + the account-level switch (operator needs
#    bedrock:PutModelInvocationLoggingConfiguration,
#    bedrock:GetModelInvocationLoggingConfiguration - the script reads the
#    config back after applying - iam:PassRole on the delivery role, and
#    kms:GetKeyPolicy for the preflight):
BEDROCK_PROMPT_LOGGING=true   # in deploy.env
scripts/deploy-observability.sh
```

Verify (after a live Claude Code session):

```
aws bedrock get-model-invocation-logging-configuration --region "$AWS_REGION"
aws logs tail "/claude/${NAME_PREFIX}/bedrock-prompts" --region "$AWS_REGION" --since 1h
aws s3 ls "s3://<BedrockPromptLogsBucketName output>/AWSLogs/" --recursive | head
```

Then note the real key layout the delivery used and tighten the bucket policy's bucket-wide `s3:PutObject` grant to those exact prefixes.

Disable: set `BEDROCK_PROMPT_LOGGING=false` and re-run `deploy-observability.sh` — the script removes the account configuration (get-then-delete, so it is a no-op when already off). The destinations stay in the stack with their data; retention/lifecycle keeps aging data out. Re-enabling later is just `=true` and a re-run — nothing is created or deleted, so the cycle is clean in both directions.

Notes & pitfalls: Enabling without the KMS prerequisite leaves Bedrock unable to write the SSE-KMS bucket — and the error blames the S3 bucket policy (see the prerequisite above). A 01 re-run only updates the key policy when the stack still **owns** the key: check `aws cloudformation describe-stacks --region "$AWS_REGION" --stack-name "$DB_STACK_NAME" --query "Stacks[0].Parameters[?ParameterKey=='KmsKeyArn'].ParameterValue" --output text` — empty means stack-managed, an ARN means BYO **or** a detached key (§11a). The CloudWatch leg only ever shows bodies ≤100 KB — an "empty-looking" log entry with an S3 reference is normal for large Claude Code contexts, not a fault. Setting `BEDROCK_PROMPT_LOGGING=false` disables invocation logging for the **whole account+region** — coordinate if anything else in the account relies on it.

11a. Re-adopting a stack-detached CMK into stack 01 (resource import)

Trigger / Frequency: One-time repair, only for deployments whose 01 stack **created** the CMK but later detached it. An earlier `deploy-database.sh` persisted the created key's ARN into `deploy.env` `KMS_KEY_ARN` and then fed it back as the `KmsKeyArn` parameter on the next re-run — CloudFormation read that as "bring-your-own", dropped the (Retain'd) `KmsKey` from the stack, **deleted the** `alias/<prefix>` **alias**, and every later `KeyPolicy` change in the template silently applied to nothing. The current script preserves ownership, so this repair applies only to deployments created before it.

Am I affected? All three of these hold:

```
# 1. the stack's parameter is non-empty (BYO mode) ...
aws cloudformation describe-stacks --region "$AWS_REGION" --stack-name "$DB_STACK_NAME" \
  --query "Stacks[0].Parameters[?ParameterKey=='KmsKeyArn'].ParameterValue" --output text
# 2. ... but the key it names was created by this stack ...
aws kms describe-key --key-id "$KMS_KEY_ARN" --region "$AWS_REGION" \
  --query 'KeyMetadata.Description' --output text
#    -> "<prefix> - data-at-rest key (RDS, secrets, logs, archives, AMP)"
#    If this is a landing-zone/central key instead: STOP - you are true BYO;
#    manage the key policy out-of-band (§11) and do not import.
# 3. ... and the stack no longer holds the KMS resources:
aws cloudformation describe-stack-resources --region "$AWS_REGION" --stack-name "$DB_STACK_NAME" \
  --query "StackResources[?LogicalResourceId=='KmsKey' || LogicalResourceId=='KmsKeyAlias'].LogicalResourceId"
#    -> []
```

Prerequisites: the **repo copy on the deploy host** must carry the ownership-preserving `deploy-database.sh` and the `DeletionPolicy: Retain` on `KmsKeyAlias` (import requires a DeletionPolicy on every imported resource). Without the script fix, the next routine 01 re-run detaches the key again.

Procedure:

```
KEY_ID="$(aws kms describe-key --key-id "$KMS_KEY_ARN" --region "$AWS_REGION" \
  --query KeyMetadata.KeyId --output text)"

# 1. Recreate the alias (deleted at detach; import can only adopt resources
#    that exist). AlreadyExistsException just means it survived - fine.
aws kms create-alias --alias-name "alias/${NAME_PREFIX}" \
  --target-key-id "$KEY_ID" --region "$AWS_REGION"

# 2. Confirm the region supports importing both types (GovCloud feature
#    sets lag; both should appear, with KeyId / AliasName identifiers).
#    CAVEAT (unconfirmed): how get-template-summary treats Condition'd
#    resources is undocumented - an empty result may mean condition
#    filtering rather than "unsupported in region". Treat empty as
#    inconclusive and let step 3's create-change-set be the real probe
#    (it fails cleanly if the type isn't importable).
aws cloudformation get-template-summary --region "$AWS_REGION" \
  --template-body file://cloudformation/01-database.yaml \
  --query "ResourceIdentifierSummaries[?ResourceType=='AWS::KMS::Key' || ResourceType=='AWS::KMS::Alias']"

# 3. IMPORT changeset - TWO PHASES. A one-shot import that also flips
#    KmsKeyArn='' fails with "As part of the import operation, you cannot
#    modify or add [Outputs]": import
#    validation compares the CONDITION-RESOLVED templates, and flipping the
#    parameter moves KmsKeyArnResolved onto its !GetAtt branch - a modified
#    Output, which imports forbid. AWS's documented resolution is exactly
#    this split: import with everything resolving as deployed, THEN update.
#
# 3a. Build the import template: the deployed template with ONLY the two
#     'Condition: CreateKmsKey' lines removed (KmsKey + KmsKeyAlias render
#     even though the stack parameter stays in BYO mode; every !If in the
#     rest of the template keeps resolving to its deployed branch). The
#     line appears exactly twice; verify before and after.
grep -c '^    Condition: CreateKmsKey$' cloudformation/01-database.yaml   # -> 2
sed '/^    Condition: CreateKmsKey$/d' cloudformation/01-database.yaml > /tmp/01-import.yaml

# 3b. IMPORT changeset: import template, EVERY parameter at its previous
#     value - including KmsKeyArn (still the key's ARN). Do NOT flip it
#     here; that is step 8.
aws cloudformation create-change-set \
  --region "$AWS_REGION" \
  --stack-name "$DB_STACK_NAME" \
  --change-set-name kms-reimport \
  --change-set-type IMPORT \
  --template-body file:///tmp/01-import.yaml \
```

```
  --parameters \
    ParameterKey=NamePrefix,UsePreviousValue=true \
    ParameterKey=VpcId,UsePreviousValue=true \
    ParameterKey=PrivateSubnetIds,UsePreviousValue=true \
    ParameterKey=DBInstanceClass,UsePreviousValue=true \
    ParameterKey=DBEngineVersion,UsePreviousValue=true \
    ParameterKey=DBAllocatedStorage,UsePreviousValue=true \
    ParameterKey=MultiAZ,UsePreviousValue=true \
    ParameterKey=BackupRetentionDays,UsePreviousValue=true \
    ParameterKey=PgauditLogClasses,UsePreviousValue=true \
    ParameterKey=PgauditLogRetentionDays,UsePreviousValue=true \
    ParameterKey=KmsKeyArn,UsePreviousValue=true \
  --resources-to-import "[
    {\"ResourceType\":\"AWS::KMS::Key\",\"LogicalResourceId\":\"KmsKey\",\"ResourceIdentifier\":{\"KeyId\":\"${KEY_ID}\"}},
    {\"ResourceType\":\"AWS::KMS::Alias\",\"LogicalResourceId\":\"KmsKeyAlias\",\"ResourceIdentifier\":{\"AliasName\":\"alias/$
{NAME_PREFIX}\"}}
  ]"
#    If this STILL reports modified resources/outputs, the deploy host's
#    01-database.yaml is not the revision the stack last deployed - diff
#    `aws cloudformation get-template` against the repo file and rebase the
#    sed edit on what the stack actually holds.

# 4. Inspect BEFORE executing - EXACTLY two rows, both action "Import"
#    (KmsKey, KmsKeyAlias). Anything else: delete-change-set, investigate.
aws cloudformation describe-change-set --region "$AWS_REGION" \
  --stack-name "$DB_STACK_NAME" --change-set-name kms-reimport \
  --query 'Changes[].[ResourceChange.Action,ResourceChange.LogicalResourceId]' --output table

# 5. Execute and wait for IMPORT_COMPLETE.
aws cloudformation execute-change-set --region "$AWS_REGION" \
  --stack-name "$DB_STACK_NAME" --change-set-name kms-reimport
aws cloudformation wait stack-import-complete --region "$AWS_REGION" \
  --stack-name "$DB_STACK_NAME"

# 6. Detect drift: import records the IMPORTED TEMPLATE's properties as the
#    resource's expected state WITHOUT comparing or applying them, so the
#    key's ACTUAL policy still holds whatever out-of-band state it had. A
#    same-template deploy-database.sh after import is a NO-OP ("no changes
#    to deploy") - it does NOT re-apply KeyPolicy; CloudFormation converges
#    the policy only on the NEXT update whose template actually changes a
#    KmsKey property. Run drift detection (the AWS-recommended post-import
#    step) to see exactly where the real key differs from the template:
aws cloudformation detect-stack-drift --region "$AWS_REGION" --stack-name "$DB_STACK_NAME"
# ... then describe-stack-resource-drifts for KmsKey once detection completes.
```

```
# 7. Confirm the Bedrock statement is on the REAL key (put there by §11's
#    out-of-band step - NOT by CloudFormation; see above):
aws kms get-key-policy --key-id "$KEY_ID" --policy-name default \
  --region "$AWS_REGION" --query Policy --output text | grep -c BedrockInvocationLogsWrite   # -> 1

# 8. Normalize back to created-key mode: the REPO template (conditions
#    restored) with KmsKeyArn flipped to '' - the change the import refused
#    to combine, legal in a normal update. The !If branches move to !GetAtt
#    but resolve to the SAME ARN, so Database/log groups compute as no-ops
#    (and the stack policy still denies Update:Replace on Database as the
#    backstop). ALLOW_KMS_PARAM_CHANGE=1 is required - the ownership guard
#    otherwise keeps the stack's recorded BYO parameter.
#
#    The empty value MUST be set IN deploy.env, NOT as a command-line env
#    var: common.sh does `source deploy.env` at load, so a prefix like
#    `KMS_KEY_ARN= scripts/deploy-database.sh` is silently OVERWRITTEN by
#    the persisted export - resolve_kms_param then sees env == stack
#    parameter (no override, no log line), CreateKmsKey stays false, and
#    the update DETACHES the freshly imported key again (the key and alias
#    physically survive - the imported template's DeletionPolicy Retain
#    governs the removal - and recovery
#    is: re-run steps 3a-5, then this step, done correctly).
( source scripts/common.sh; set_env_var KMS_KEY_ARN "" )
ALLOW_KMS_PARAM_CHANGE=1 scripts/deploy-database.sh
#    EXPECT the log line "ALLOW_KMS_PARAM_CHANGE=1: overriding ... -> ''"
#    before the deploy starts - if it is absent, STOP: the flip is not
#    happening and the run will detach the key again. On success the
#    script's tail re-persists KMS_KEY_ARN=<ARN> into deploy.env from
#    KmsKeyArnResolved (it does so whenever the env value is empty). Do
#    the blank + deploy in one sitting: while deploy.env holds an empty
#    KMS_KEY_ARN, ensure_ecr_repo would create non-CMK repos.

# 9. Confirm the flip took, then one plain re-run proves the fixed script
#    now preserves ownership (a no-op update leaving the parameter empty):
grep '^export KMS_KEY_ARN=' scripts/deploy.env    # -> the key ARN again
aws cloudformation describe-stack-resources --region "$AWS_REGION" \
  --stack-name "$DB_STACK_NAME" \
  --query "StackResources[?starts_with(LogicalResourceId,'KmsKey')].LogicalResourceId"
#    -> ["KmsKey", "KmsKeyAlias"]
scripts/deploy-database.sh
```

> **The template becomes the source of truth again on the next real `KeyPolicy` change — not at import time.** Import doesn't touch the key's actual policy, and a same-template update is a no-op; but the next template edit that alters `KmsKey` (e.g. a future key-policy statement) replaces the FULL policy, silently removing any out-of-band statement not in `01-database.yaml`. `BedrockInvocationLogsWrite` is safe (it ships in the template);

> anything key admins added while the key was detached must be folded into the template's `KeyPolicy` or expect it gone on that update. Check with step 6's drift results before moving on.

---

12. Client recovery — Claude Code won't start after `/logout`

Trigger / Frequency: A developer reports that `claude` exits at launch with **"Unable to connect to Anthropic services"**, and `claude auth login` refuses with *"forceLoginMethod is 'gateway' in managed settings; run interactive /login to authenticate."* Typically right after they ran `/logout`, or on a first-ever run on a clean profile.

Why it happens (this deployment makes a benign check fatal): the two logout paths are **not** equivalent.

- `/logout` **(slash command)** clears credentials **and** onboarding (`clearOnboarding: true`): it sets `hasCompletedOnboarding = false` in `%USERPROFILE%\.claude.json` and **deletes the whole credential store** — including `enterpriseGateway` (the session) *and* `gatewayTrust` (the pinned TLS fingerprint).
- `claude auth logout` **(CLI subcommand)** clears credentials only (`clearOnboarding: false`) and leaves the client able to start.

After `/logout`, the next launch re-enters the **onboarding** flow. Claude Code derives the login method from *whether gateway credentials exist* — with them deleted it resolves to `firstParty`, which flips OAuth onboarding on and puts a **connectivity preflight first in the step list**. That preflight is absent while a gateway session exists, which is why the failure only appears after a logout. It `GET`s `https://api.anthropic.com/api/hello` **and** `https://platform.claude.com/v1/oauth/hello`, requires **HTTP 200 on both**, and on failure prints the error and exits 1 — **before** the gateway login screen is ever drawn. Those two hosts are fixed in the build: the gateway URL never substitutes for them. On a gateway-only egress path they are unreachable by design, so the developer is deadlocked — the CLI dies before login, and `claude auth login` is barred by the managed policy.

Confirming it is this and not a gateway outage (a non-200 — including a Zscaler block page — is the failure):

```
curl.exe -sS -o NUL -w "%{http_code}`n" https://api.anthropic.com/api/hello
curl.exe -sS -o NUL -w "%{http_code}`n" https://platform.claude.com/v1/oauth/hello
```

Steps (run as the affected developer; no admin rights needed): restore the onboarding flag, which removes the preflight step from the startup path.

```
$p = "$env:USERPROFILE\.claude.json"
Copy-Item $p "$p.bak"          # this file also holds project history — back it up
$j = Get-Content $p -Raw | ConvertFrom-Json
if ($j.PSObject.Properties.Name -contains 'hasCompletedOnboarding') {
  $j.hasCompletedOnboarding = $true
} else {
  $j | Add-Member -NotePropertyName hasCompletedOnboarding -NotePropertyValue $true
```

```
}
$j | ConvertTo-Json -Depth 100 | Set-Content $p -Encoding utf8
```

Then open a **new** terminal, run `claude`, and run `/login` — the locked "Cloud gateway" screen appears with the URL pre-filled (§3 of `client-config.md`).

Verification: `claude` starts to a prompt instead of exiting; `/login` completes the Okta round-trip; `/model` lists only the three served models (runbook 6).

Notes & pitfalls:

- **The TLS fingerprint prompt comes back.** `/logout` deleted `gatewayTrust` along with the session, so the next connect is a fresh trust-on-first-use. The developer must confirm it against the fingerprint IT published (runbook 1) — do not let them click through it, since that prompt is the control that detects TLS interception in front of the gateway FQDN.
- `forceRemoteSettingsRefresh` **is not the culprit** and does not need removing. Its startup gate short-circuits as *succeeded* while logged out (remote managed settings only apply once a gateway session exists), so it does not block recovery. Resist the temptation to strip it from the GPO during triage — that would silently drop the model allowlist fleet-wide.
- **Opening** `api.anthropic.com` / `platform.claude.com` **in Zscaler also "fixes" it** and is the wrong lever: it widens egress permanently to work around a one-line local config, against the gateway-only posture the deployment is built on.
- **This is not user error to be trained away** — `/logout` is a documented, discoverable command. Treat recurrence as expected and keep this runbook reachable by the service desk.

Prevention: tell developers to sign out with `claude auth logout`, never `/logout`; that path leaves onboarding intact, so `claude` still starts and `/login` reconnects. The same trap applies to a **fresh install** on a locked-down laptop — a first-ever run has `hasCompletedOnboarding` unset and takes the identical preflight path — so exercise a never-used profile before any broad rollout rather than discovering it per-user.

Rollback / recovery: the edit is a single boolean in a backed-up file; restore `.claude.json.bak` to undo. Nothing server-side changes.

---

13. Teardown

Trigger / Frequency: Decommissioning the deployment (lab cleanup or end of life). Rare and deliberate.

Order is the reverse of deploy: `04 and 03 → 02 → 01`. The portal (`04`) and observability (`03`) stacks both import from `02` and are independent of each other — delete them (in either order, or in parallel) before the gateway. **03 owns no ECS service or Cloud Map namespace** (the collector is a sidecar in the 02 gateway task), so its delete is simple — no lingering collector ENIs, no discovery-service-before-namespace ordering to wait on. There is intentionally **no teardown script**; delete stacks explicitly so each destructive step is a conscious act. Downstream stacks import upstream exports, so an out-of-order delete fails on the export lock.

Preconditions & the protection layers you must clear first:

- **RDS** `DeletionProtection: true` blocks deleting the DB stack — disable it first (a stack update setting it false, or the console), and expect a **final snapshot** (`DeletionPolicy: Snapshot`).
- **ALB** `deletion_protection.enabled: true` blocks deleting the gateway stack — disable it first.
- The **stack policies** set by the deploy scripts deny `Update:Replace` / `Update:Delete` on `LoadBalancer` / `Database` during *updates*; they do not block `delete-stack`, but the deletion-protection flags above do. Clear those flags before deleting.

Steps (exact commands):

```
# 4) Download portal (if deployed)
aws cloudformation delete-stack --region "$AWS_REGION" --stack-name "$PORTAL_STACK_NAME"
aws cloudformation wait stack-delete-complete --region "$AWS_REGION" --stack-name "$PORTAL_STACK_NAME"

# 3) Observability
aws cloudformation delete-stack --region "$AWS_REGION" --stack-name "$OBS_STACK_NAME"
aws cloudformation wait stack-delete-complete --region "$AWS_REGION" --stack-name "$OBS_STACK_NAME"

# 2) Gateway (disable ALB deletion protection first, then delete)
aws cloudformation delete-stack --region "$AWS_REGION" --stack-name "$GATEWAY_STACK_NAME"
aws cloudformation wait stack-delete-complete --region "$AWS_REGION" --stack-name "$GATEWAY_STACK_NAME"

# 1) Database (disable RDS deletion protection first; a final snapshot is taken)
aws cloudformation delete-stack --region "$AWS_REGION" --stack-name "$DB_STACK_NAME"
aws cloudformation wait stack-delete-complete --region "$AWS_REGION" --stack-name "$DB_STACK_NAME"
```

What survives deletion (verified against the templates' `DeletionPolicy`):

- **KMS CMK** (`KmsKey`, 01) — `Retain`. Everything at rest was encrypted with it; retained so retained data stays readable. Schedule key deletion manually only after all encrypted artifacts are gone. **Redeploy consequence:** `deploy.env` still holds this key's ARN in `KMS_KEY_ARN` (persisted by `deploy-database.sh`), and a fresh deploy would consume it as **bring-your-own** — 01 would reference the retained key but never manage its policy again (`deploy-database.sh` warns loudly on this path). Decide deliberately: clear `KMS_KEY_ARN` so the new stack creates and manages a new key, or keep it and own the key policy out-of-band from then on.
- **ALB access-logs bucket** (`AlbLogsBucket`, 02) — `Retain`.
- **Activity-archive bucket** (`ActivityArchiveBucket`, 03) — `Retain` (CMK-encrypted).
- **AMP workspace** (`Workspace`, 03) — `Retain`.
- **Portal artifacts bucket** (`ArtifactsBucket`, 04) — `Retain` (CMK-encrypted; holds the published release binaries).
- **Every CloudWatch log group, in every stack** — `Retain` (a deliberate decision: logs outlive stacks; the cfn-guard gate `log_groups_survive_teardown` enforces it). This covers the ECS task groups (gateway — which now also holds the collector sidecar's `otel` streams — grafana, portal), the activity

window, the portal download-audit group, the RDS `postgresql`/pgaudit export group (`/aws/rds/instance/${NAME_PREFIX}-store/postgresql`, pre-created by 01 with the CMK), and the db-admin Lambda groups (pre-created by 02 with the CMK). Deleting a retained group afterwards is a deliberate manual act. Note the redeploy consequence: the groups carry fixed names, so a later **re-create collides** with the retained groups and the new stack fails — export what you need, then delete them first (`greenfield-deployment.md` Phase 1 has the command list). The three pre-created groups (RDS postgresql and the two db-admin Lambdas) additionally collide in an account where those services already auto-created them under the same names.

- **RDS instance** (`Database`, 01) — `DeletionPolicy: Snapshot` → a **final snapshot** is taken; the running instance is removed. The snapshot persists.
- Everything else (ALB, ECS services/cluster, secrets, VPC endpoints, Lambdas, Firehose) is **deleted** with its stack.

Verification: all three `stack-delete-complete` waits return; `aws cloudformation describe-stacks` reports the stacks gone; confirm the retained buckets, CMK, AMP workspace, and final DB snapshot still exist if you intend to keep them, or clean them up explicitly.

Rollback / recovery: Redeploy from scratch per `greenfield-deployment.md`. Data recovery relies on the retained final RDS snapshot (restore per runbook 8) and the retained buckets.

Notes & pitfalls: Deleting a stack that another stack still imports from fails — always `04 and 03 → 02 → 01`, waiting for each delete to complete before the next tier. Two waits to expect: the db-admin Lambda ENIs can linger ~20 minutes attached to 01's db-client SG (a 01 delete failing with a dependency violation usually just needs a retry), and named Secrets Manager secrets enter a 7–30 day recovery window — to redeploy the same `NAME_PREFIX` immediately, first `aws secretsmanager delete-secret --force-delete-without-recovery` the `<prefix>/*` secrets. Retained resources are **not** free; account for the retained buckets, CMK, AMP workspace, and snapshots after teardown.